CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: Budget Buddy

Prepared By: Casas, Elissa Mae Buenvenida

Version: 2.0

Date: February 28, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	[Date]	[Your Name]	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

Budget Buddy is a personal finance management application that enables users to track expenses, manage budgets, and monitor savings goals. A key feature is **Shared Expenses**, allowing groups such as roommates, families, or friends to split costs and track balances collaboratively. The system includes a backend API, a responsive web application, and a mobile app, all integrated to provide seamless financial tracking across platforms.

1.2 Objectives

1. Develop a fully functional finance app with authentication, expense tracking, budget management, and shared expenses.
2. Implement a layered architecture with backend API, web, and mobile clients.
3. Create RESTful APIs for communication between all system components.
4. Design a responsive interface with widgets for quick access.
5. Deploy all system components to production-ready environments.

1.3 Scope

Included Features:

- User registration and authentication (email/password, Google OAuth)
- Expense tracking (CRUD operations, categories)
- Budget management (set limits, monitor usage)
- Shared expenses (group creation, splitting costs, balance tracking)
- Reports and analytics (summaries, charts)
- File upload (receipts linked to expenses)
- Email notifications (welcome, budget alerts)

Excluded Features:

- Real payment gateway integration (outside sandbox mode)
- Advanced financial forecasting
- Multi-language support
- Push notifications

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Budget Buddy system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Budget Buddy
Domain: Personal Finance Management
Primary Users:

1. Students
2. Professionals
3. Families
4. Groups

Problem Statement: Users need a simple way to track expenses and split costs without complex financial tools.

Solution: A minimalist finance app focusing on expense tracking, budgeting, and shared group expenses.

2.2 Core User Journeys

Journey 1: Individual Expense Tracking

1. User logs in.

2. Adds a new expense with category and amount.

3. Views updated dashboard with expense summary.

Journey 2: Shared Expenses

1. User creates a shared group (e.g., “Roommates”).

2. Adds shared expense (e.g., rent).

3. App splits cost among members.

4. Each member sees their share and balance.

Journey 3: Budget Management

- User sets monthly budget limit.
- Tracks spending against budget.
- Receives alert when nearing limit.

2.3 Feature List (MoSCoW)

MUST HAVE

- User authentication (register, login, logout, JWT)
- Expense tracking (CRUD operations)
- Budget management (set limits, monitor usage)
- Shared expenses (group creation, splitting costs)
- Dashboard with summaries

SHOULD HAVE

- Reports and analytics (charts, summaries)
- Notifications for overspending or due shared expenses
- File upload for receipts

COULD HAVE

- Exportable reports (CSV/PDF)
- Multi-account support
- Savings goal tracking

WON'T HAVE

- Real payment processing (outside sandbox)
- Bank account linking
- Advanced forecasting

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Expense Tracking

- **Screens:** Expense Entry, Expense List
- **Functions:** Add expense, edit expense, delete expense, categorize expense
- **Persistence:** Database storage per user
- **API endpoints:** GET /expenses, POST/expenses. PUT/expenses/ {id}, DELETE/expenses/{id}

Feature: Budget Management

- **Screens:** Budget Setup, Budget Overview
- **Functions:** Set monthly/weekly budget, track usage, receive alerts
- **Persistence:** Database storage per user
- **API Endpoints:** GET /budgets, POST /budgets, PUT /budgets/{id}, DELETE /budgets/{id}

Feature: Shared Expenses

- **Screens:** Group Creation, Shared Expense List, Balance Overview
- **Functions:** Create group, add shared expense, split costs among members, view balances
- **Persistence:** Database storage with group-user mapping
- **API Endpoints:** POST /groups, GET /groups/{id}, POST /groups/{id}/expenses, GET /groups/{id}/balances

Feature: Reports & Analytics

- **Screens:** Reports Dashboard, Charts View
- **Functions:** View spending summaries, category breakdowns, monthly comparisons
- **API Endpoints:** GET /reports, GET /reports/{id}

Feature: Admin Panel

- **Screens:** User Management, Expense Overview, Group Management
- **Functions:** Add/edit/delete users, view all expenses and groups
- **API Endpoints:** Admin-prefixed endpoints with role validation

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid email and strong password
- ➢ And confirm password matches

- And click "Create Account"
- Then my account should be created
- And I should be automatically logged in
- And redirected to the homepage

AC-2: Expense Tracking Flow

- Given I am logged in as a user
- When I add a new expense with valid details
- And save the expense
- Then the expense should appear in my expense list
- And my dashboard should update with the new total

AC-3: Shared Expenses Flow

- Given I am logged in as a user
- When I create a group and add a shared expense
- And assign members to split the cost
- Then each member should see their share in the balance overview
- And the group expense should be stored in the database

AC-4: Budget Management Flow

- Given I am logged in as a user
- When I set a monthly budget limit
- And add expenses that approach or exceed the limit
- Then I should receive an alert notification
- And the dashboard should reflect the budget status

AC-5: Admin Management Flow

- Given I am logged in as an administrator
- When I add or edit a user or group record
- And save the changes
- Then the updated information should appear in the admin panel
- And be accessible to authorized users only

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms
- Real-time updates for shared expenses groups

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication for protected routes
- Password hashing with Bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection for web and mobile inputs
- Rate limiting: 100 requests/minute per IP
- Role-based access control (Admin vs. Regular User)
- /me endpoint to fetch current authenticated user

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 34+ (Android 14)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+
- **Backend:** Spring Boot (Java 17+), Spring Security, JPA/Hibernate
- **Database:** PostgreSQL or MySQL (normalized schema, minimum 5 tables)

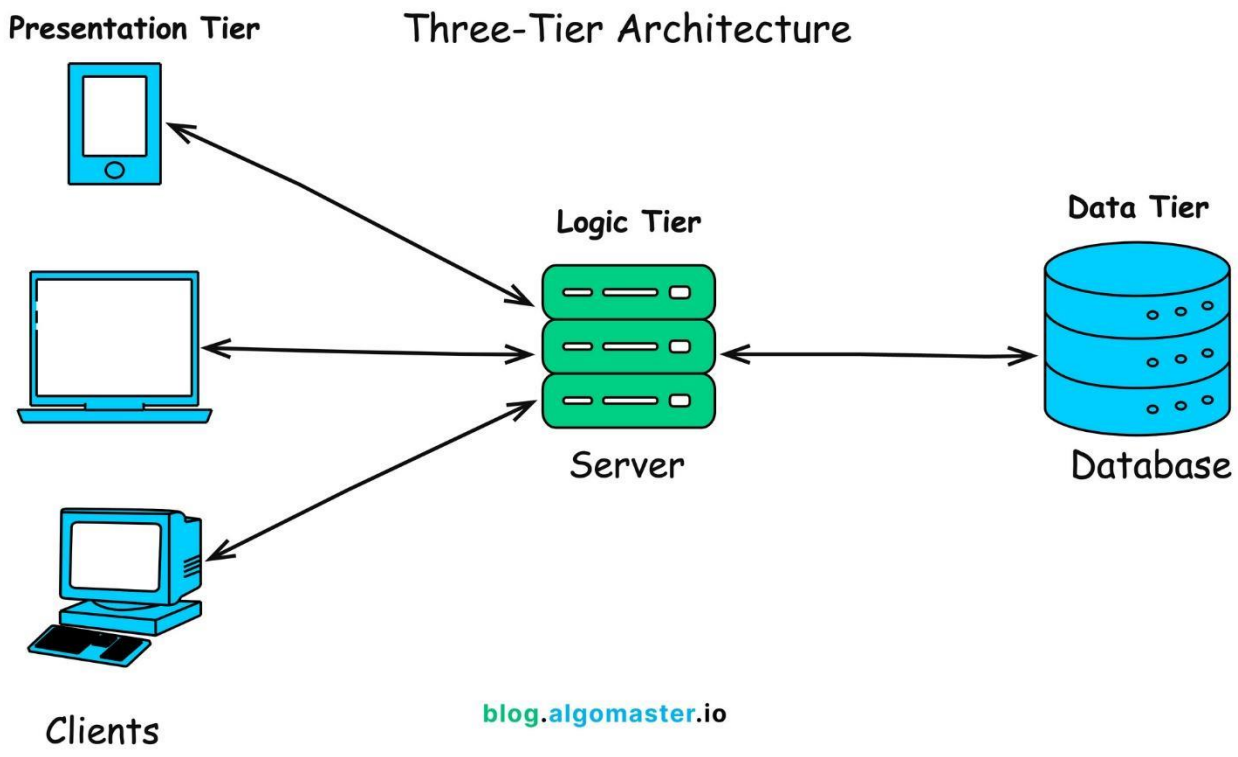
3.4 Usability Requirements

- Expense entry should be completed within 30 seconds for new users
- WCAG 2.1 Level AA compliance for accessibility
- Consistent navigation across all screens (web and mobile)
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile
- Keyboard navigation support for desktop users
- Role-aware UI (Admin vs. Regular User)
- Mobile app must consume the same backend API (no mock data allowed)

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": "StrongPassword123",
	"firstname": "John",
	"lastname": "Doe",
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": 1,
	"email": "johndoe@gmail.com",
	"firstname": "John",
	"lastname": "Doe"
	},
	"accessToken": "jwt_token_here",
	"refreshToken": "refresh_token_here"
	},
	"error": null,

	"timestamp": "2026-02-28T11:30:00Z" }

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: “johndoe@gmail.com”,
	“password”: “StrongPassword123”,
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": 1,
	"email": "johndoe@gmail.com",
	"role": "USER"
	},
	"accessToken": "jwt_token_here",
	"refreshToken": "refresh_token_here"
	},
	"error": null,
	"timestamp": "2026-02-28T11:30:00Z"
	}

User Logout

Description	User Logout
API URL	/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)

Response Structure	{
	"success": true,
	"data": {
	"message": "Logged out successfully"
	},
	"error": null,
	"timestamp": "2026-02-28T11:30:00Z"
	}

Get Current User

Description	Get Current User (/me)
API URL	/auth/me
HTTP Method	GET
Authentication	Bearer Token (JWT)

Successful Response	{
	"success": true,
	"data": {
	"user": {
	"id": 1,
	"email":"johndoe@gmail.com",
	"firstname": "John",
	"lastname": "Doe",

	"role": "USER" } }, "error": null, "timestamp": "2026-02-28T11:30:00Z" }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1: Invalid Login Credentials	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2026-02-28T15:40:00Z"
	}

Example 2: Validation Failed (Expense Entry)	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"amount": "Amount must be greater than 0",
	"category": "Category is required"
	}
	},
	"timestamp": "2026-02-28T15:41:00Z"
	}

Example 3: Budget Exceeded	{
	"success": false,
	"data": null,
	"error": {
	"code": "BUSINESS-001",
	"message": "Budget exceeded",

	"details": "Adding this expense exceeds your monthly budget limit" }, "timestamp": "2026-02-28T15:42:00Z" }

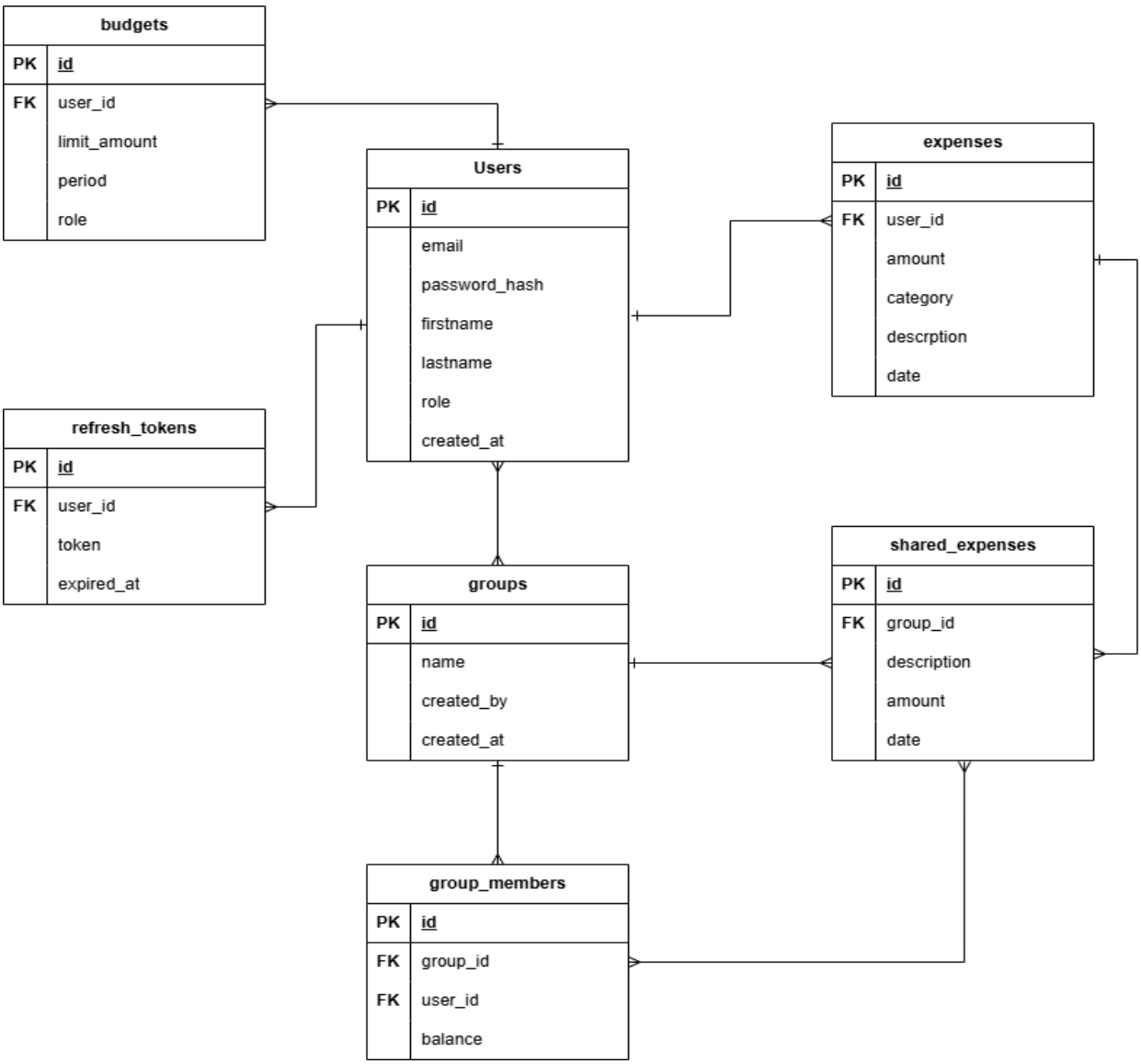
Example 4: Resource not found (Expense ID)	{
	"success": false,
	"data": null,
	"error": {
	"code": "DB-001",
	"message": "Resource not found",
	"details": "Expense with ID 123 does not exist"
	},
	"timestamp": "2026-02-28T15:43:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **Users ↔ Expenses (One-to-Many)**
  - Each user can record multiple expenses.
  - Each expense belongs to exactly one user.
  - Foreign Key: expense.user_id → users.id
- **Users ↔ Budgets (One-to-Many)**
  - Each user can set multiple budgets (e.g., monthly, weekly).
  - Each budget belongs to exactly one user.

    - Foreign Key: budgets.user_id → users.id
- **Users ↔ Groups (Many-to-Many)**
    - A user can join multiple groups (e.g., "Roommates", "Family").
    - A group can have multiple users.
    - Join Table**:** group_members with user_id and group_id.
        - Foreign Keys:
        - group_members.user_id → users.id
        - group_members.group_id → groups.id
- **Groups → SharedExpenses (One-to-Many)**
    - Each group can have multiple shared expenses.
    - Each shared expense belongs to exactly one group.
    - Foreign Key: shared_expenses.group_id → groups.id
- **GroupMembers → SharedExpenses (Indirect Relationship)**
    - Shared expenses are split among group members.
    - Balance field in group_members tracks each member's share of group expenses.
- **Users → RefreshTokens (One-to-Many)**
    - Each user can have multiple refresh tokens for secure authentication.
    - Each token belongs to exactly one user.
    - Foreign Key: refresh_tokens.user_id → users.id

Key Tables:

1. **users** – User accounts and authentication
2. **expenses** – Individual expense records per user
3. **budgets** – Budget limits set by users
4. **groups** – Shared expense groups (e.g., roommates, family)
5. **group_members** – Join table mapping users to groups with balances
6. **shared_expenses** – Expenses linked to groups
7. **refresh_tokens** – JWT refresh tokens for authentication

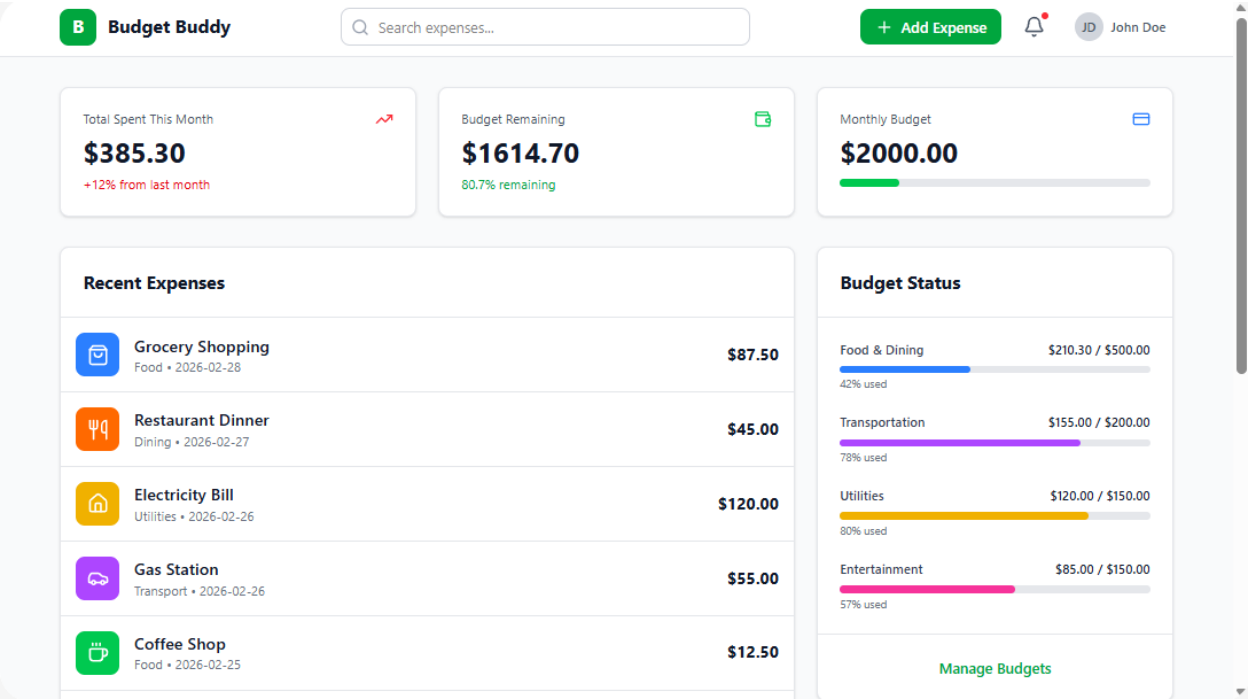
Table Structure Summary:

- **users**: id, email, password_hash, firstname, lastname, role, created_at
- **expenses**: id, user_id, amount, category, description, date, receipt_file
- **budgets**: id, user_id, limit_amount, period, created_at
- **groups**: id, name, created_by, created_at
- **group_members**: id, group_id, user_id, balance
- **shared_expenses**: id, group_id, amount, description, date
- **refresh_tokens:** id, user_id, token, expires_at

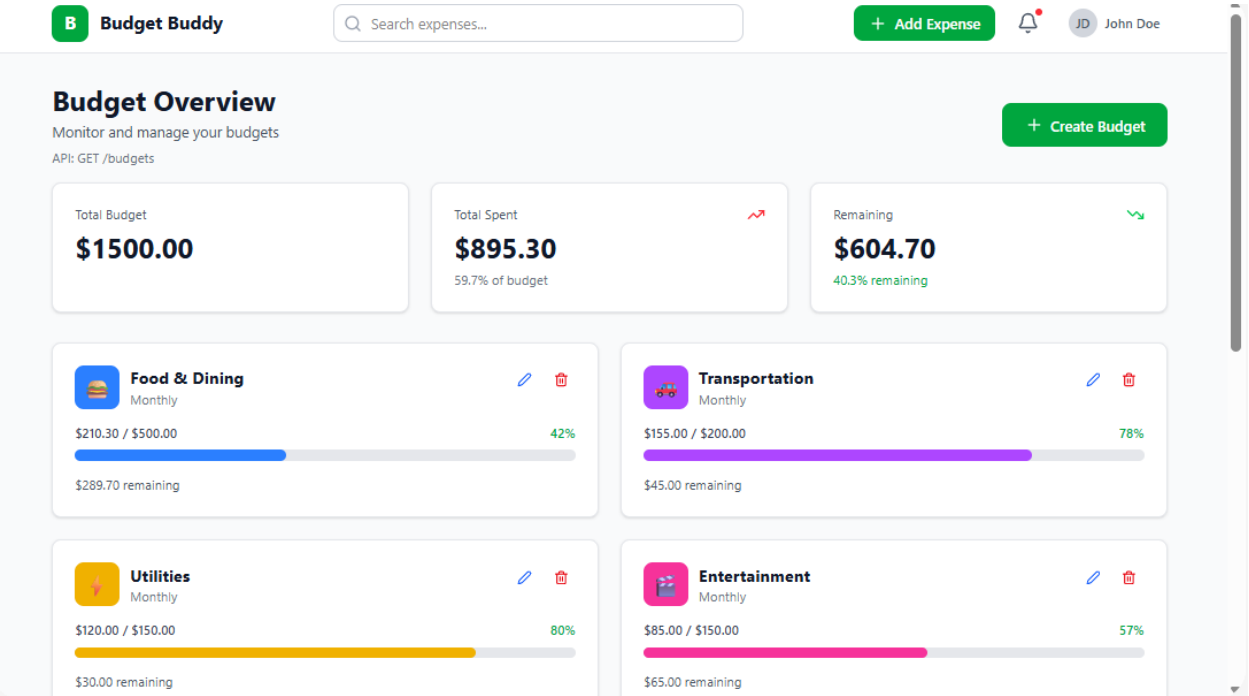
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

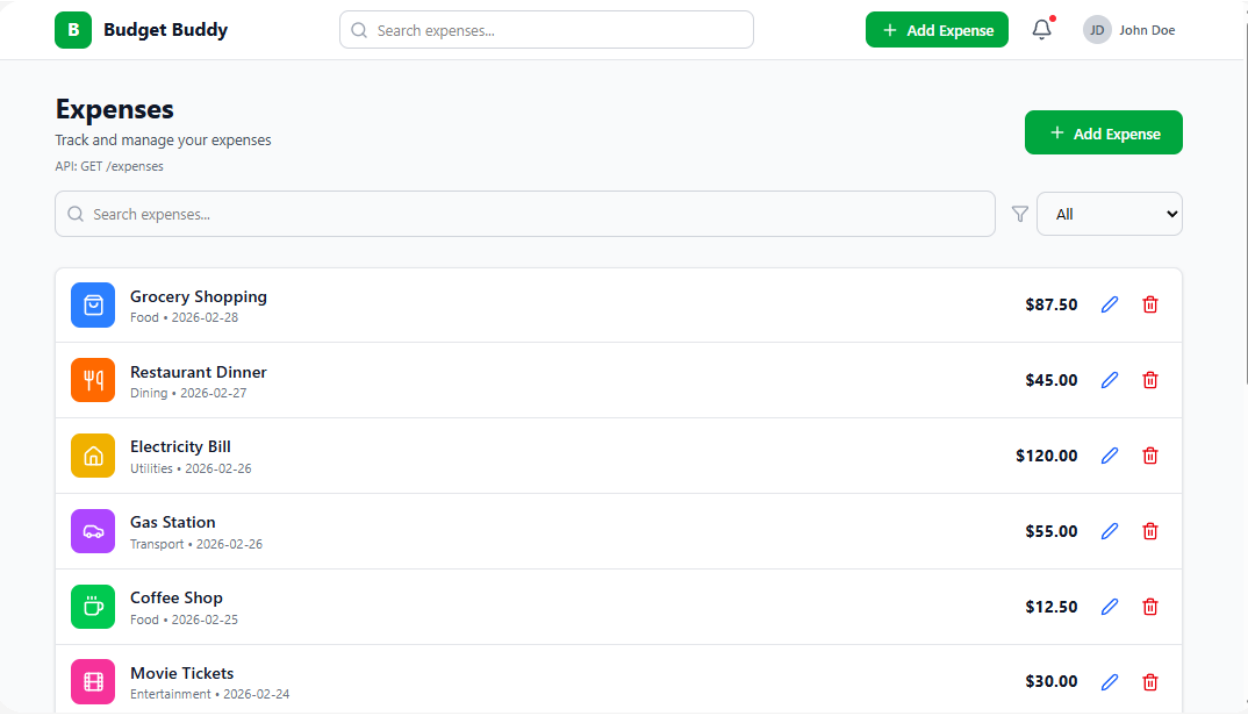
Homepage



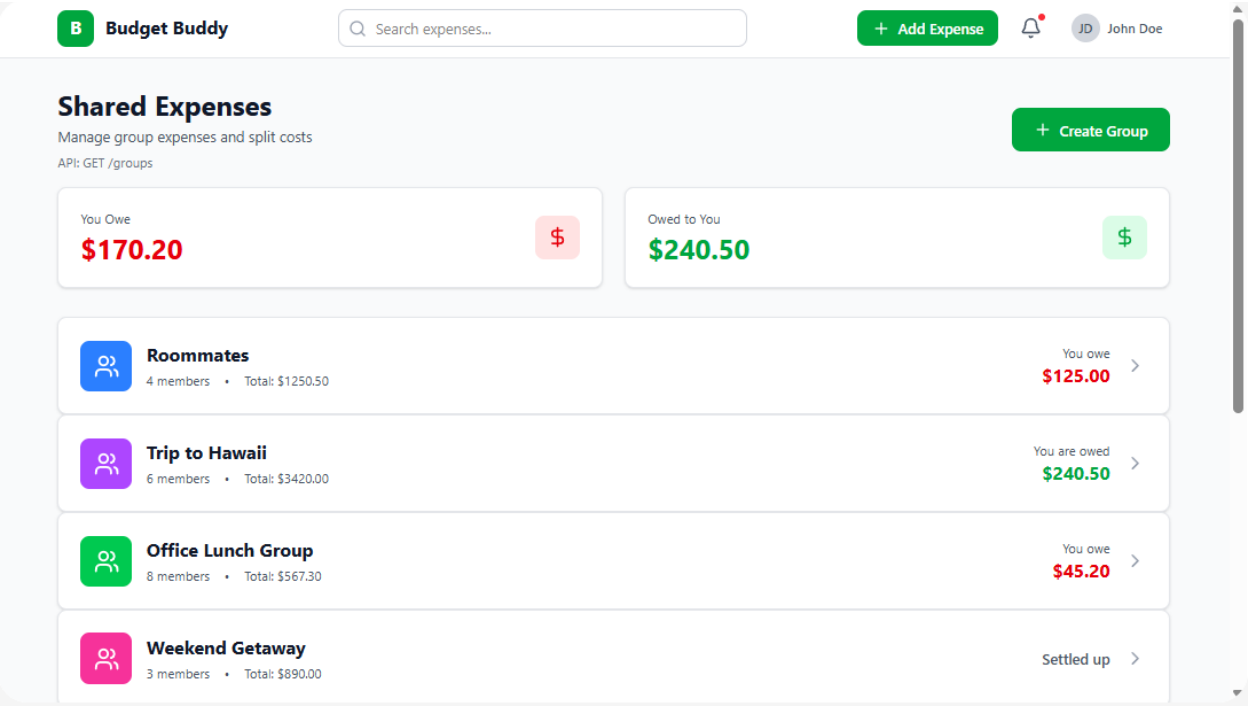
Budget Overview Page



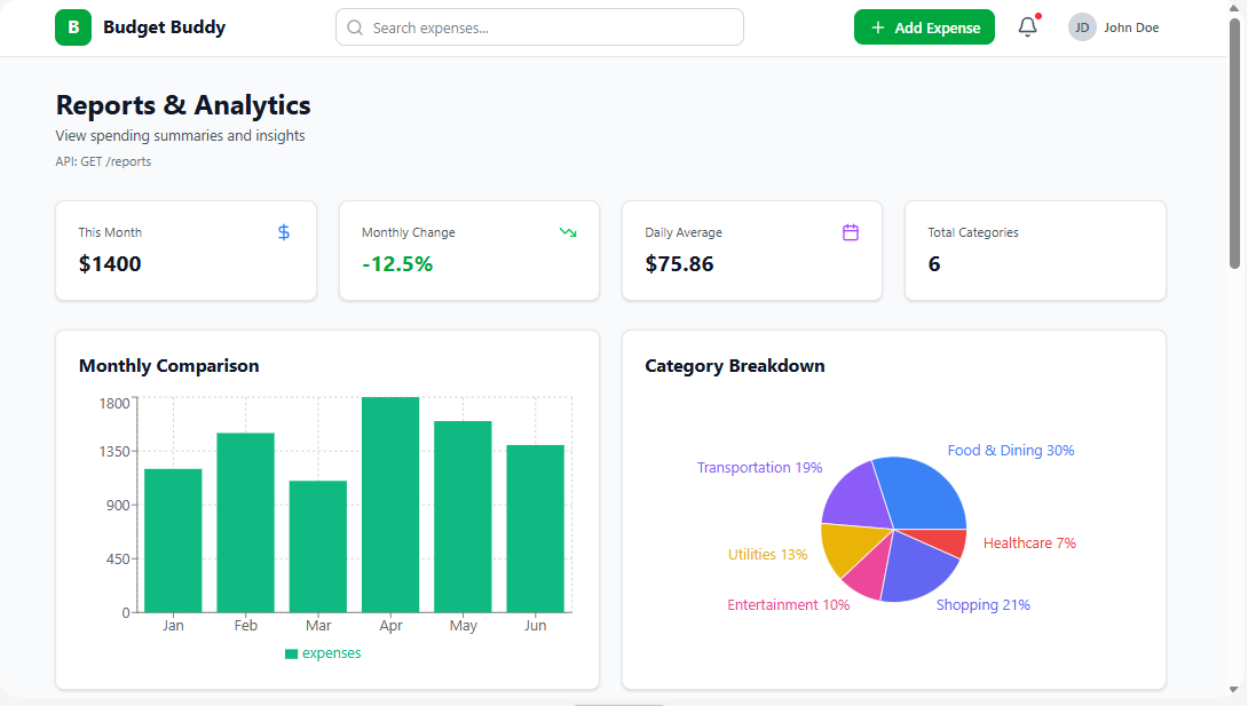
Expenses Page



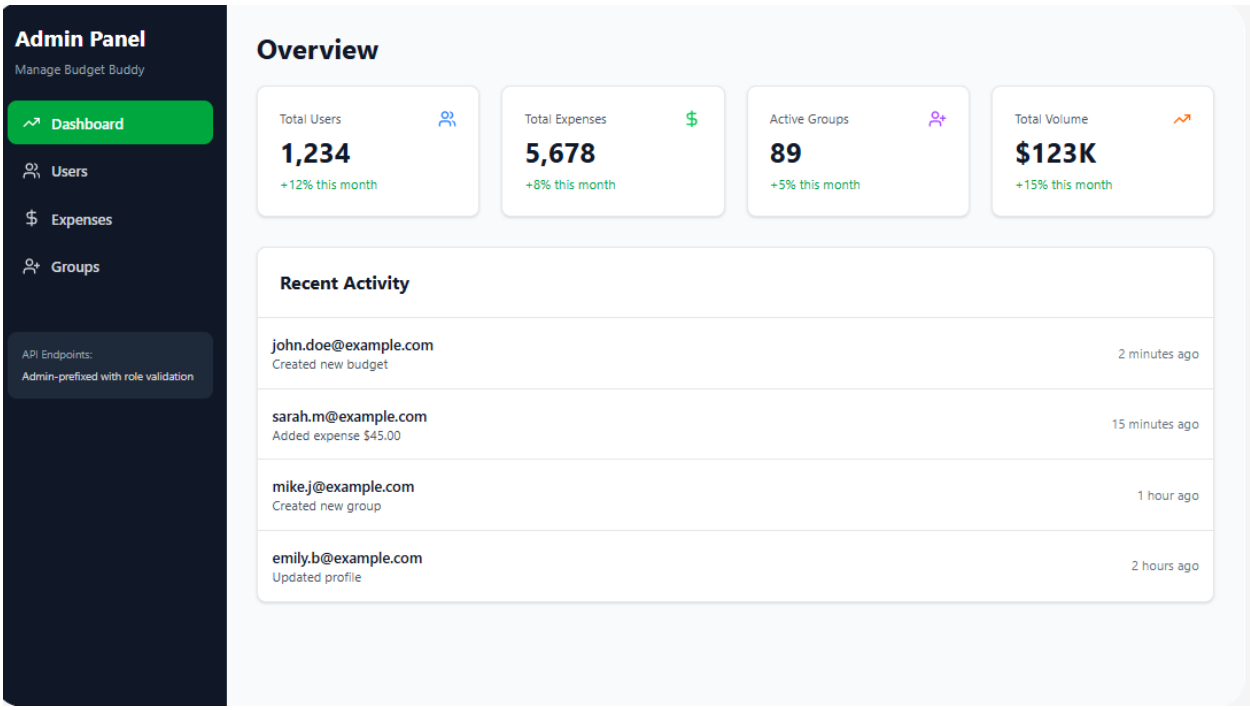
Shared Expenses Page



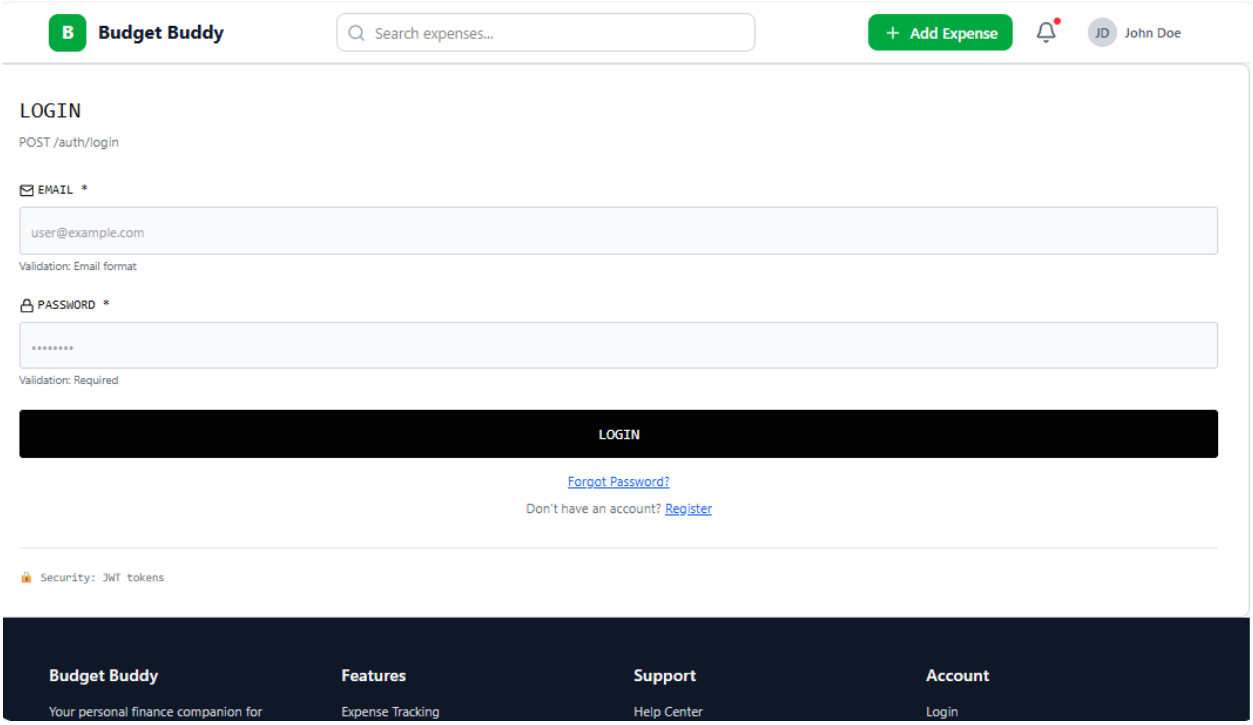
Reports & Analystics Page


B
Budget Buddy
Search expenses...
+ Add Expense
JD
John Doe
Reports & Analytics
View spending summaries and insights
API: GET /reports
This Month
$1400
Monthly Change
-12.5%
Daily Average
$75.86
Total Categories
6
Monthly Comparison
1800
1350
900
450
0
Jan
Feb
Mar
Apr
May
Jun
expenses
Category Breakdown
Food & Dining 30%
Transportation 19%
Healthcare 7%
Utilities 13%
Entertainment 10%
Shopping 21%

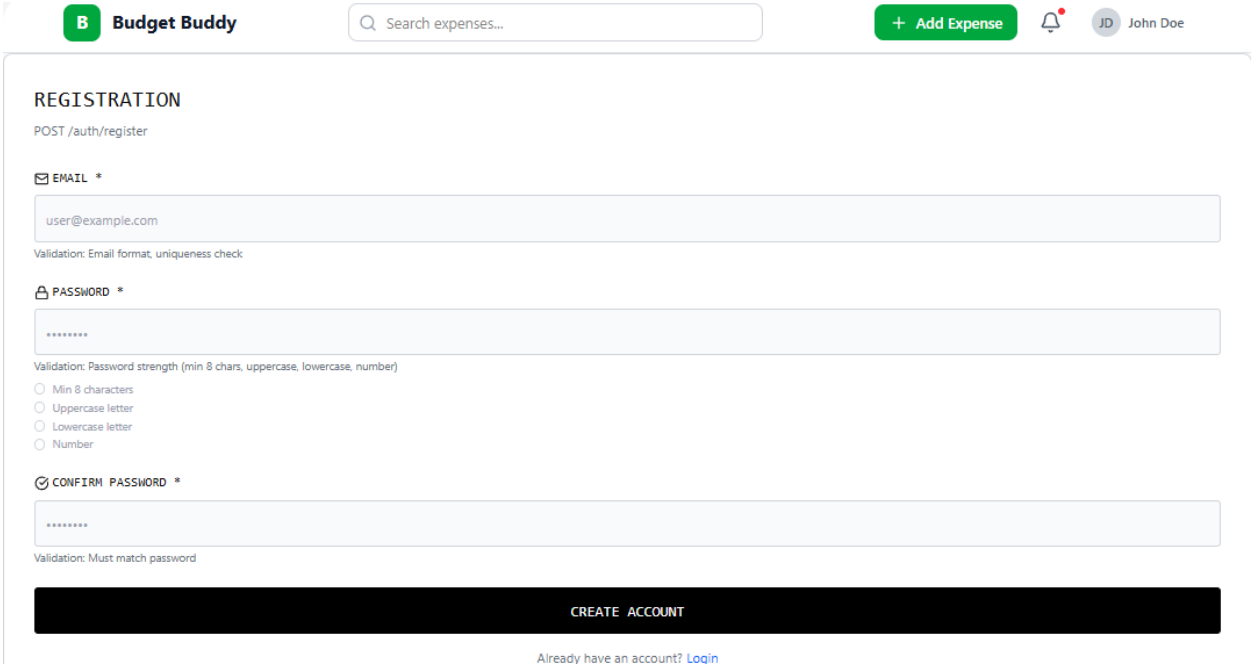
Admin Panel



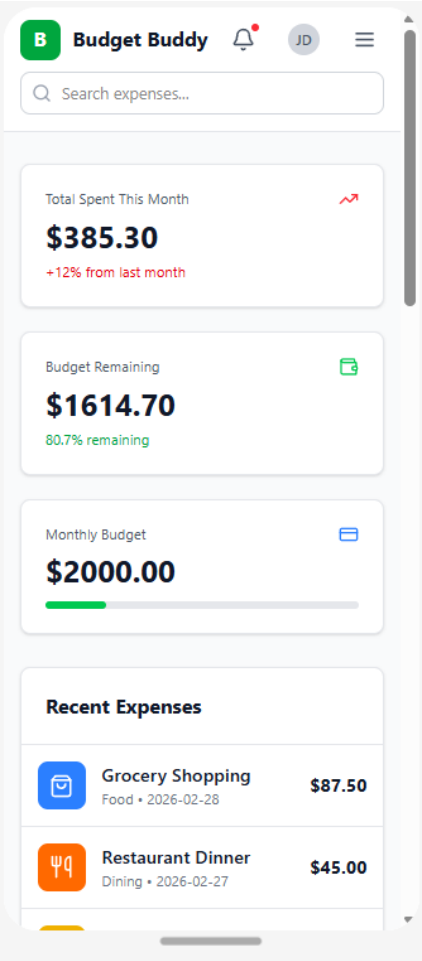
Login Page

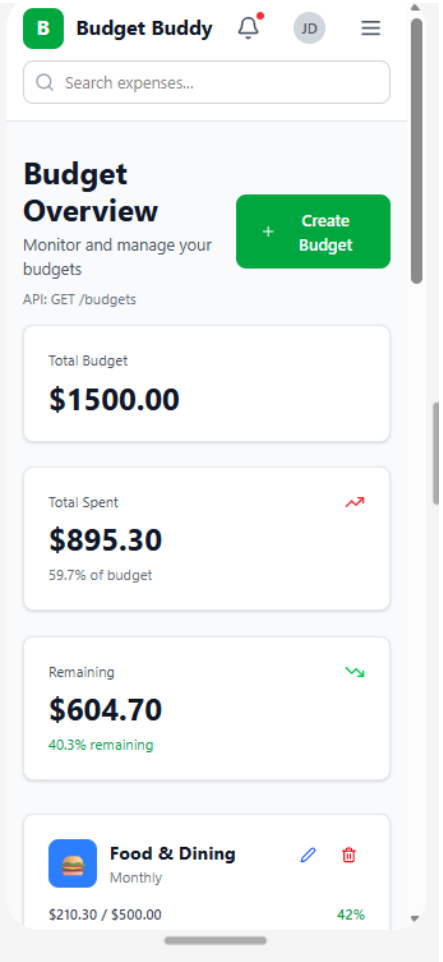


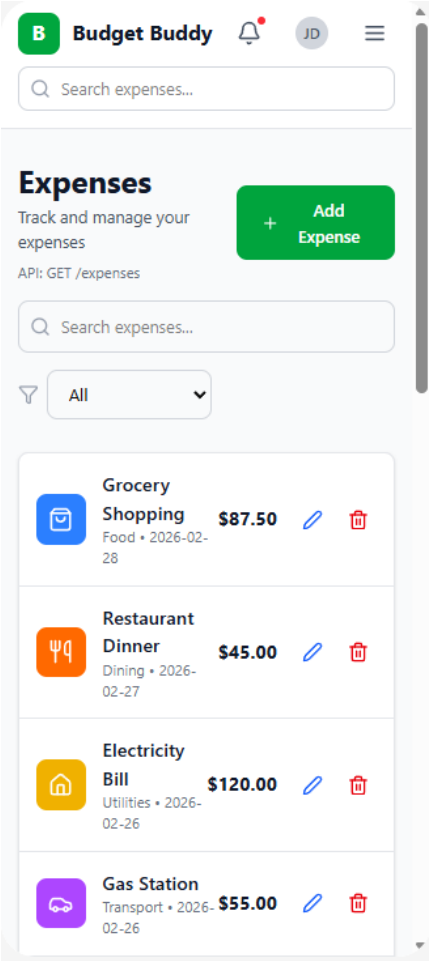
Registration Page

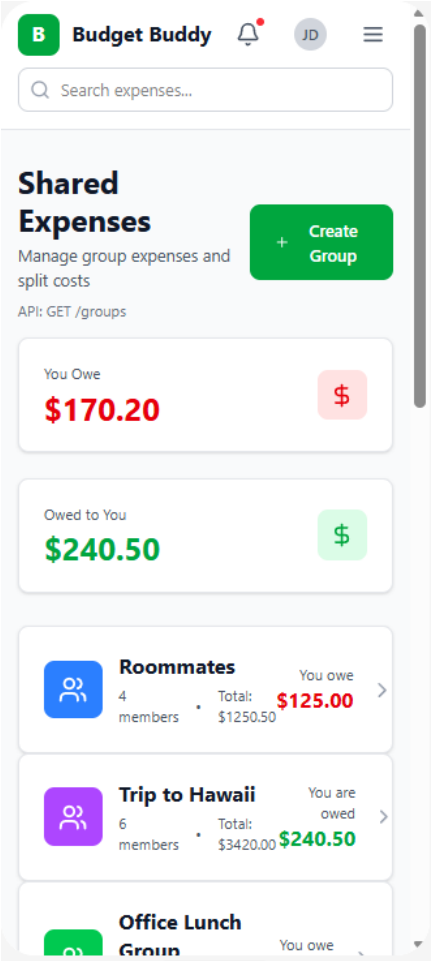


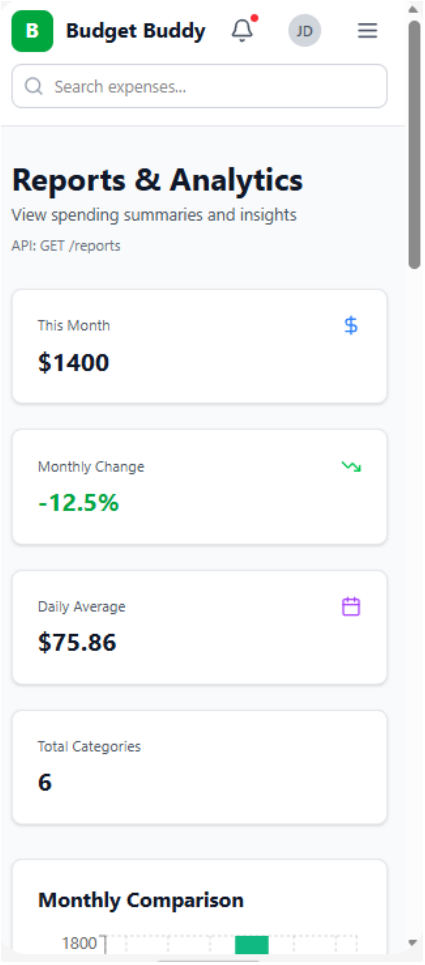
7.2 Mobile Application Wireframes

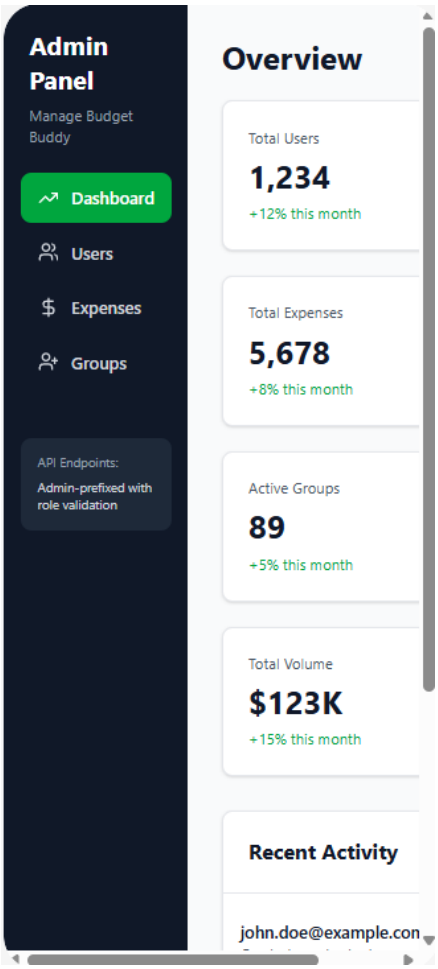

B
Budget Buddy
JD
Search expenses...
Total Spent This Month
$385.30
+12% from last month
Budget Remaining
$1614.70
80.7% remaining
Monthly Budget
$2000.00
Recent Expenses
Grocery Shopping
Food • 2026-02-28
$87.50
Restaurant Dinner
Dining • 2026-02-27
$45.00


B
Budget Buddy
JD
Search expenses...
Budget Overview
Monitor and manage your budgets
API: GET /budgets
Create Budget
Total Budget
$1500.00
Total Spent
$895.30
59.7% of budget
Remaining
$604.70
40.3% remaining
Food & Dining
Monthly
$210.30 / $500.00
42%


B
Budget Buddy
JD
Search expenses...
Expenses
Track and manage your expenses
API: GET /expenses
+ Add Expense
Search expenses...
All
Grocery Shopping
Food • 2026-02-28
$87.50
Restaurant Dinner
Dining • 2026-02-27
$45.00
Electricity Bill
Utilities • 2026-02-26
$120.00
Gas Station
Transport • 2026-02-26
$55.00


B
Budget Buddy
JD
Search expenses...
Shared Expenses
Manage group expenses and split costs
API: GET /groups
+ Create Group
You Owe
$170.20
$
Owed to You
$240.50
$
Roommates
4 members • Total: $1250.50
You owe
$125.00
Trip to Hawaii
6 members • Total: $3420.00
You are owed
$240.50
Office Lunch Group
You owe





Mobile-Specific Features:

- **Touch-optimized buttons (min 44x44px):** All interactive elements (e.g., "Add Expense", "Create Budget", edit/delete icons) meet the minimum tap target size for mobile usability.
- **Gesture support (swipe, pull-to-refresh):**
  - Swipe-to-delete is implemented in the expense list.
  - Pull-to-refresh is available on the budget overview and expense history screens.
- **Offline caching for product images:**
  - Expense icons and category images are cached locally to ensure smooth offline access and reduce data usage.
- **Bottom navigation for main actions:**

    - Mobile app uses a fixed bottom nav bar with icons for: Home, Search, Add Expense, Groups, and Profile.

- **Simplified forms for mobile input:**
    - Expense and budget forms use dropdowns, numeric inputs, and auto-focus fields.
    - Minimal typing required; optimized for one-handed use.

Design System:

- **Colors:**
    - Primary: #2563EB (blue)
    - Secondary: #7C3AED (purple)
    - Success: #10B981 (green)
    - Error: #EF4444 (red)

- **Typography:**
    - Font: Inter
    - Responsive sizing:
        - Mobile: 14–16px
        - Desktop: 16–18px
        - Headings scale based on screen width

- **Spacing:**
    - 8px grid system used across all layouts
    - Consistent padding/margin for cards, buttons, and inputs

- **Components:**
    - Reusable UI elements:
        - Buttons (primary, secondary, icon-only)
        - Input fields (text, number, date)
        - Cards (expense, budget, group)
        - Modals (confirmation, alerts)

- **Responsive:**
    - Mobile-first design
    - Breakpoints:

        - 640px (mobile)
        - 768px (tablet)
        - 1024px (desktop)
    - Layouts adapt fluidly across devices

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete

- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements